
```
MergeSort(item A[ ], HASH smaller, int start , int end )
```

```
if start < end then
    int mid = (start + end)/2
    MergeSort(A, smaller, start, mid)
    MergeSort(A, smaller, mid + 1, end)
    Merge(A, smaller, start, end, mid)
```

```
Merge(item A[], HASH smaller, int start , int end , int mid )
```

```
int i, j, k, h
int count = 0
i = start
j = mid + 1
k = start
while i ≤ mid and j ≤ end do
    if A[i] ≤ A[j] then
        B[k] = A[i]
        int old = iff(smaller.lookup(A[i]) == nil, 0, smaller.lookup(A[i]))
        smaller.insert(A[i], old + count)
        i = i + 1
    else
        count = count + 1
        B[k] = A[j]
        j = j + 1
    k = k + 1
j = end
for h = mid downto i do
    A[j] = A[h]
    int old = iff(smaller.lookup(A[h]) == nil, 0, smaller.lookup(A[h]))
    smaller.insert(A[h], old + 1)
    j = j - 1
for j = start to k - 1 do
    A[j] = B[j]
```

Soluzione $O(n \log n)$, basata su alberi red-black Menziono questa soluzione perché interessante, ma non fornisco codice perché richiede parecchie modifiche agli alberi redblack. La soluzione consiste nel mantenere in ogni nodo x dell'albero il numero di nodi dell'albero contenuti nel sottoalbero radicato in x . In un albero binario è facilmente implementabile inserendo un contatore in ogni nodo e aumentandolo di uno tutte le volte che si attraversa un nodo durante le operazioni di inserimento. Sfortunatamente, in un albero redblack bisogna anche modificare tutte le operazioni di bilanciamento e questo è troppo complesso.

Assumendo di avere questi valori, quando si inserisce un valore il numero di valori che sono più piccoli di quello è pari a zero se viene inserito come figlio sinistro del parent, è pari al numero di nodi nel sottoalbero sinistro del parent più uno (il parent stesso). La procedura di inserimento potrebbe restituire questo valore oltre all'albero. A questo punto, si inseriscono i valori a partire dall'ultimo: quando viene inserito il valore, il numero di nodi minori è pari al valore restituito dall'operazione insert.

Esercizio B1 – Corsa a ostacoli – Punti ≥ 8

Questo problema può essere risolto tramite programmazione dinamica. Scriviamo un'equazione di ricorrenza che restituisca il numero di percorsi per andare dalla cella (i, j) alla cella $(1, 1)$.

$$DP[i][j] = \begin{cases} 0 & A[i][j] = \text{false} \vee i == 0 \vee j = 0 \\ 1 & i = 1 \wedge j = 1 \wedge A[i][j] = \text{true} \\ DP[i-1][j] + DP[i][j-1] & \text{altrimenti} \end{cases}$$

In altre parole, il numero di percorsi che passano per una cella con valore **false** è 0; il numero di percorsi che "sfiorano" in alto e a sinistra è 0; il numero di percorsi a partire dalla cella $(1, 1)$ è pari a 1; il numero di percorsi a partire da qualunque altra cella è pari al numero di percorsi andando a destra e in basso.

```

int paths(boolean [] A, int n)
{
    int[] DP = newint[0...n][0...n] = {0}
    for i = 1 to n do
        for j = 1 to n do
            if not A[i][j] then
                | DP[i][j] = 0
            else if i == 1 and j == 1 then
                | DP[i][j] = 1
            else
                | DP[i][j] = DP[i-1][j] + DP[i][j-1]
        }
    }
    return DP[n][n]
}

```

Complessità $O(n^2)$, derivante dai due cicli.

Esercizio B2 – Multi core – Punti ≥ 10

Ordine crescente Dimostriamo che scorrendo i task in ordine crescente non si minimizza il tempo di esecuzione totale. Si considerino task di lunghezza $[1, 1, 2]$ e $k = 2$ processori. I task vengono divisi fra i due processori nel modo seguente $[1, 2]$ e $[1]$ con tempo di esecuzione totale pari a 4, mentre la suddivisione $[1, 1]$ e $[2]$ ha tempo di esecuzione totale pari a 2.

Ordine decrescente Dimostriamo che scorrendo i task in ordine decrescente non si minimizza il tempo di esecuzione totale. Si considerino task di lunghezza $[3, 3, 2, 2, 2]$ e $k = 2$ processori. I task vengono divisi fra i due processori nel modo seguente $[3, 2]$ e $[3, 2, 2]$ con tempo di esecuzione totale pari a 7, mentre la suddivisione $[3, 3]$ e $[2, 2, 2]$ ha tempo di esecuzione totale pari a 6.

Ordine qualsiasi Ovviamente, una soluzione che accetta gli elementi in ordine qualsiasi non può funzionare, perché potrebbe ricevere gli input dei due casi precedenti, nei quali l'algoritmo non restituisce il valore ottimale richiesto.

Note di correzione Quasi tutti hanno trovato controesempi per i primi due scenari. Sul terzo, alcuni hanno tentato di dimostrare la correttezza, perdendosi in alcuni passaggi. Questo è accettabile, perché è possibile commettere errori. Altri, invece, hanno osservato che partendo da uno degli esempi del primo o del secondo scenario, il terzo scenario si comportava meglio, concludendo erroneamente che sia un algoritmo ottimo. Questo non è accettabile, perché un concetto importante che dovrebbe esservi rimasto dal corso è che non si può dimostrare la correttezza di un algoritmo solo basandosi su esempi in cui l'algoritmo funziona bene. Per questo motivo, il loro voto è stato abbassato.

Esercizio B3 – Magic square – Punti ≥ 12

Per risolvere questo problema, dobbiamo utilizzare la tecnica del backtracking per generare tutte le possibili permutazioni dei numeri $1 \dots n^2$ all'interno del quadrato, per poi verificare che sia effettivamente un quadrato magico ed eventualmente stamparlo. Modifichiamo quindi l'algoritmo per generare tutte le permutazioni.

La funzione principale crea la matrice A di dimensione $n \times n$ che conterrà i valori del quadrato magico e un insieme *left*, che inizialmente contiene tutti gli elementi compresi fra 1 e n^2 . Viene poi chiamata la funzione ricorsiva *pmRec()*, che riempie gli elementi a partire dalla cella $(1, 1)$, selezionando i valori ancora da piazzare dall'insieme *left*.

Particolare attenzione deve essere prestata all'ordine di riempimento. La funzione richiama se stessa incrementando l'indice j , quello delle colonne. Quando l'indice j supera n , viene incrementato l'indice delle righe. Se anche questo supera n , la matrice è stata riempita (abbiamo realizzato una permutazione) e va controllata tramite la funzione *isMagic()*; in caso positivo, viene stampata. Algoritmi simili a quanto appena descritto prendono 100%. Una possibile potatura, mostrata in rosso nel codice, deriva dal fatto che noi possiamo sapere a priori qual è il valore della somma delle righe, colonne e diagonali: è pari alla somma di tutti i numeri da 1 a n^2 , diviso per il numero di righe e colonne.

$$\frac{1}{n} \cdot \sum_{i=1}^{n^2} i = \frac{1}{n} \cdot \frac{n^2 \cdot (n^2 + 1)}{2} = \frac{n \cdot (n^2 + 1)}{2}$$

Al completamento di ogni riga, possiamo quindi verificare che il valore ottenuto dalla somma della riga ($sumSoFar(A[i])$ per semplificare) sia pari a questo valore, in caso negativo la ricerca lungo questo ramo viene pota-

```

printMagic(int n)


---


    int[][] A = new int[1...n][1...n]
    SET left = Set()
    for i = 1 to n2 do
        | left.insert(i)
    pmRec(A, n, 1, 1, left)


---


pmRec(int[][] A, int n, int i, int j, SET left)


---


    if j > n then
        | if sumSoFar(A[i]) ≠ n · (n2 + 1)/2 then
        | | return
        j = 1
        i = i + 1
        if i > n then
            | if isMagic(A, n) then
            | | print A
            | return
    SET choices = left.copy()
    foreach k ∈ choices do
        | A[i][j] = k
        | left.remove(k)
        | pmRec(A, n, i, j + 1, left)
        | left.insert(k)

```

Senza considerare la potatura, l'algoritmo genera tutte le permutazioni dei numeri fra 1 e n^2 , che sono $(n^2)!$. Al di là delle stampe che sono relativamente rare, la copia dell'insieme *left* necessaria per evitare problemi con il **foreach** richiede tempo $O(n^2)$ e quindi l'algoritmo è $O(n^2(n^2)!)$. Alcune soluzioni proposte dagli studenti erano basate su insiemi *left* implementati come vettori booleani di dimensione n^2 , il che costringeva poi a fare un ciclo di costo $O(n^2)$ su tutti i valori per verificare il prossimo elemento da scegliere, con costo comunque $O(n^2)$.